1. Euclidean Distance:

- This is the most common distance measure used in clustering.
- Euclidean distance can be used for data with continuous variables, such as height, weight, or age.
- if *p = (p1, p2,…, pn)* and *q = (q1, q2,…, qn)* are two points in Euclidean *n*-space, then the distance (*d*) from *p* to *q*, or from *q* to *p* is given by the Pythagorean formula:

$$\mathrm{d}(\mathbf{p},\mathbf{q}) = \mathrm{d}(\mathbf{q},\mathbf{p}) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + \cdots + (q_n - p_n)^2}$$

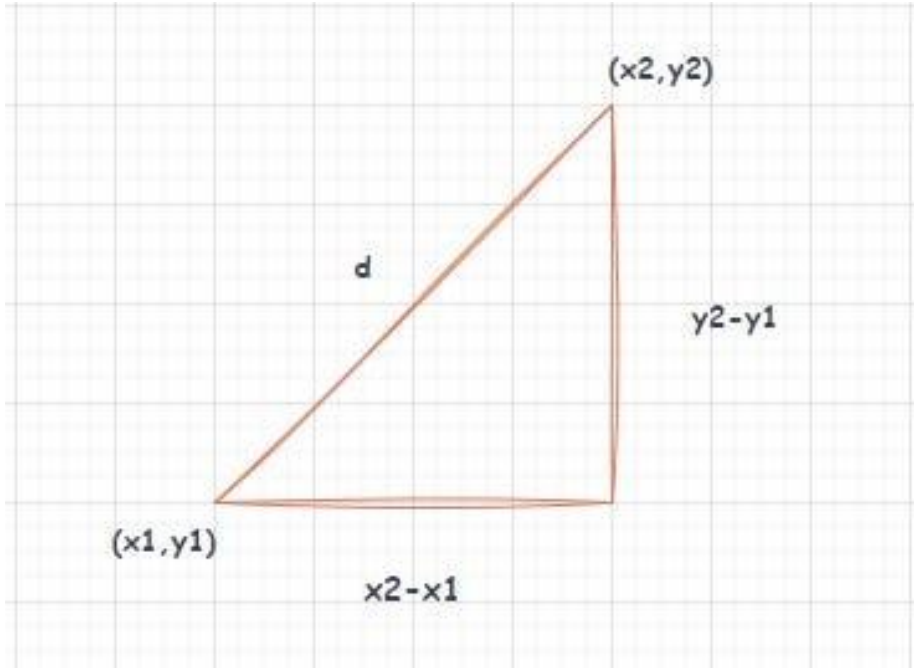
$$= \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}.$$

Calculate Euclidean distance using Math Import function:

```
import math

def euclidean_distance(point1, point2):
# Calculate the sum of the squared differences between the
coordinates
squared_diffs = [(point1[i] - point2[i]) ** 2 for i in
range(len(point1))]
sum_squared_diffs = sum(squared_diffs)
# Take the square root of the sum to get the Euclidean distance
distance = math.sqrt(sum_squared_diffs)
return distance

point1 = (1, 2, 3)
point2 = (4, 5, 6)
distance = euclidean_distance(point1, point2)
print(distance)
```



Output: 5.196152422706632

2. **Manhattan Distance:**

- This distance measure is also known as city block distance or taxicab distance.

- The taxicab name refers to the shortest path that a taxicab would take between city blocks (coordinates on the grid).

```
from math import sqrt

#create function to calculate Manhattan distance
def manhattan(a, b):
return sum(abs(val1-val2) for val1, val2 in zip(a,b))

#define vectors
A = [2, 4, 4, 6]
B = [5, 5, 7, 8]

#calculate Manhattan distance between vectors
manhattan(A, B)
```

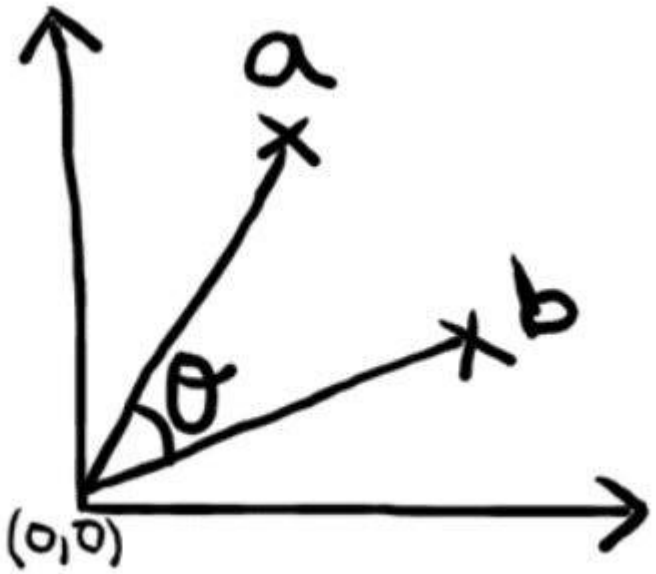
Output: 9

Manhattan Distance Formula

$$D_{mn}(x_i, x_j) = \sum_{l=1}^{d} |x_{il} - x_{jl}|$$

3. **Cosine Similarity:**

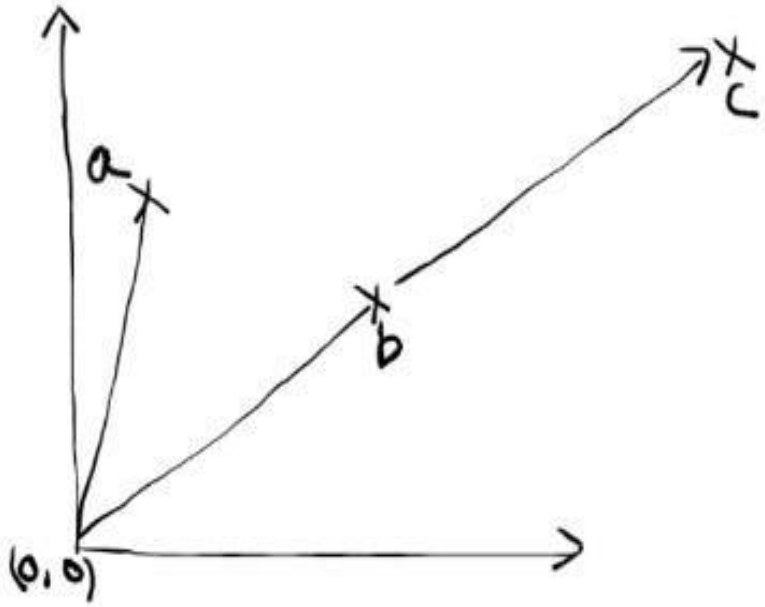
- This similarity measure is commonly used for text-based data or other high-dimensional data.
- It measures the cosine of the angle between two vectors in a multidimensional space.
- The cosine similarity ranges from -1 to 1, with 1 indicating that the vectors are identical a value of 0 indicating that they are orthogonal (perpendicular) to each other.



$$\cos\theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$$

3. Cosine Similarity

- Cosine similarity cares only about the angle between the two vectors and not the distance between them. Assume there's another vector c in the direction of b.
- Now, the cosine similarity between b and c is 1 since the angle between b and c is 0 and cos(0) = 1.
- Even though the distance between b and c is large compared to a and b cosine similarity cares only about the direction of the vector and not the distance.



```
def cos_sim(a, b):
"""Takes 2 vectors a, b and returns the cosine similarity according
to the definition of the dot product
"""
dot_product = np.dot(a, b)
norm_a = np.linalg.norm(a)
norm_b = np.linalg.norm(b)
return dot_product / (norm_a * norm_b)

# the counts we computed above
sentence_m = np.array([1, 1, 1, 1, 0, 0, 0, 0, 0])
sentence_h = np.array([0, 0, 1, 1, 1, 1, 0, 0, 0])
sentence_w = np.array([0, 0, 0, 1, 0, 0, 1, 1, 1])

# We should expect sentence_m and sentence_h to be more similar
print(cos_sim(sentence_m, sentence_h)) # 0.5
print(cos_sim(sentence_m, sentence_w)) # 0.25
```

Output: 0.5, 0.25

K-means clustering – implementation

	CustomerID	Genre	Age	Annual.Income..k..	Spending.Score..1.100.
1	1	Male	19	15	39
2	2	Male	21	15	81
3	3	Female	20	16	6
4	4	Female	23	16	77
5	5	Female	31	17	40
6	6	Female	22	17	76
7	7	Female	35	18	6
8	8	Female	23	18	94
9	9	Male	64	19	3
10	10	Female	30	19	72
11	11	Male	67	19	14
12	12	Female	35	19	99

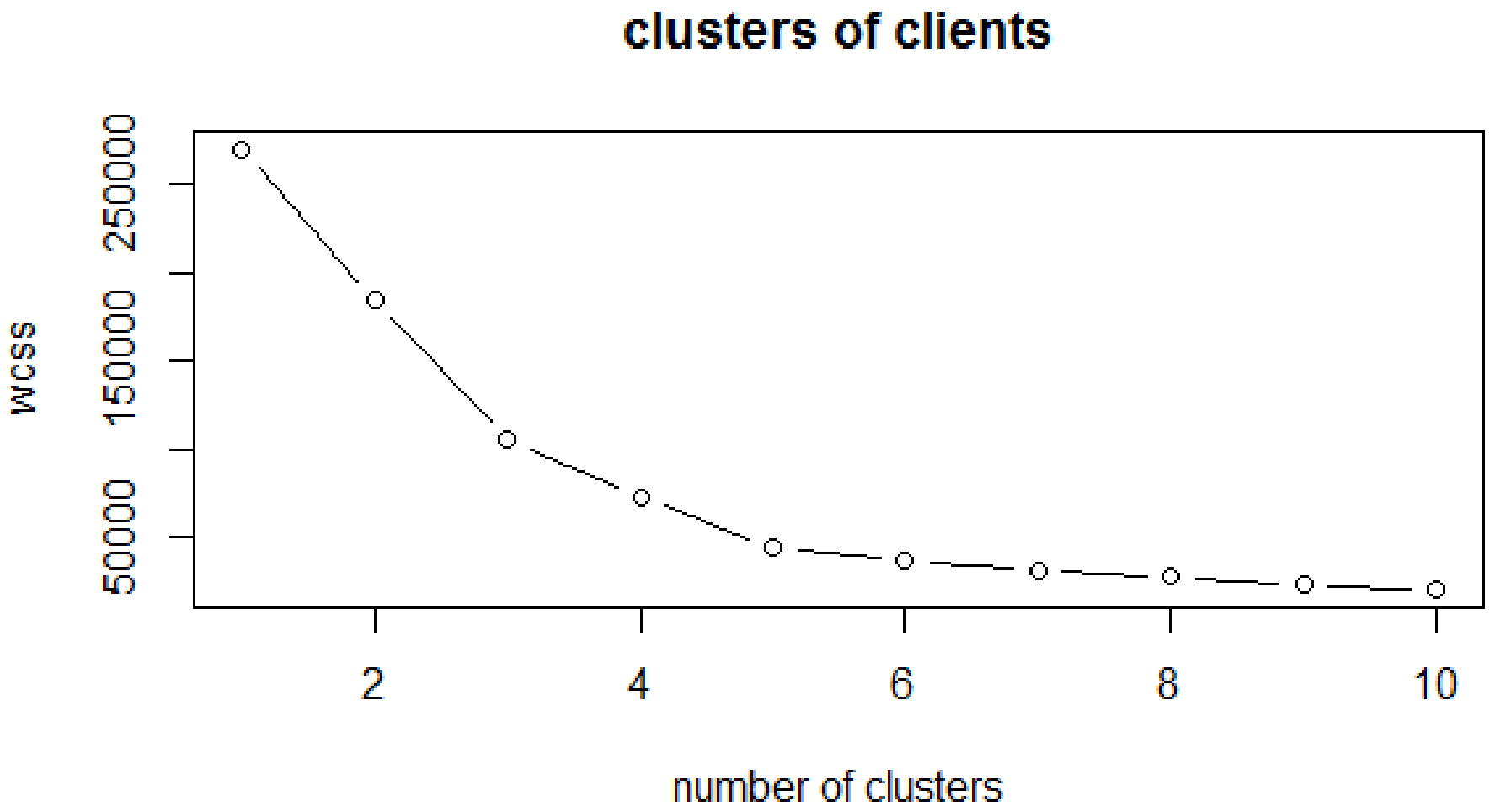
Showing 1 to 12 of 200 entries

K-means clustering – implementation

	Annual.Income..k..	Spending.Score..1.100.
1	15	39
2	15	81
3	16	6
4	16	77
5	17	40
6	17	76
7	18	6
8	18	94
9	19	3
10	19	72
11	19	14
12	19	99

Showing 1 to 12 of 200 entries

K-means clustering – implementation



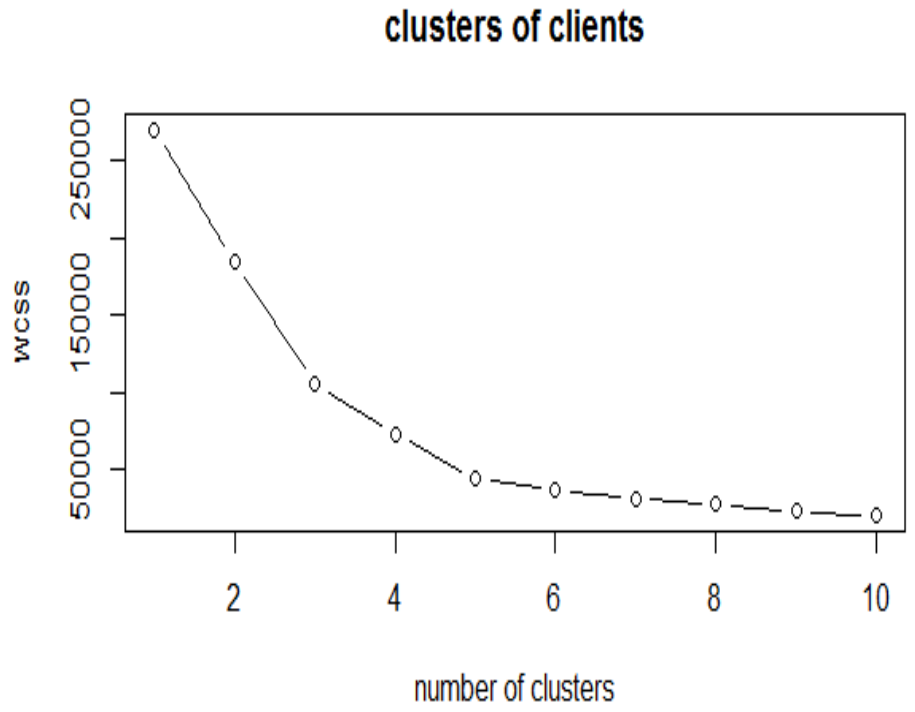
K-means clustering in python

```
# Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Importing the mall dataset with pandas
dataset = pd.read_csv('mall.csv')
X = dataset.iloc[:, [3, 4]]. values
```

K-means clustering in python

```
#Using the elbow method to find the optimal number of clusters
from sklearn.cluster import Kmeans
wcss = []
for i in range (1, 11):
    kmeans = KMeans(n_clusters = i, init = 'k-means++', max_iter = 300,
                                                    n_init = 10, random_state = 0)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)
plt.plot(range(1, 11), wcss)
plt.title('The elbow Method')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.show()
```

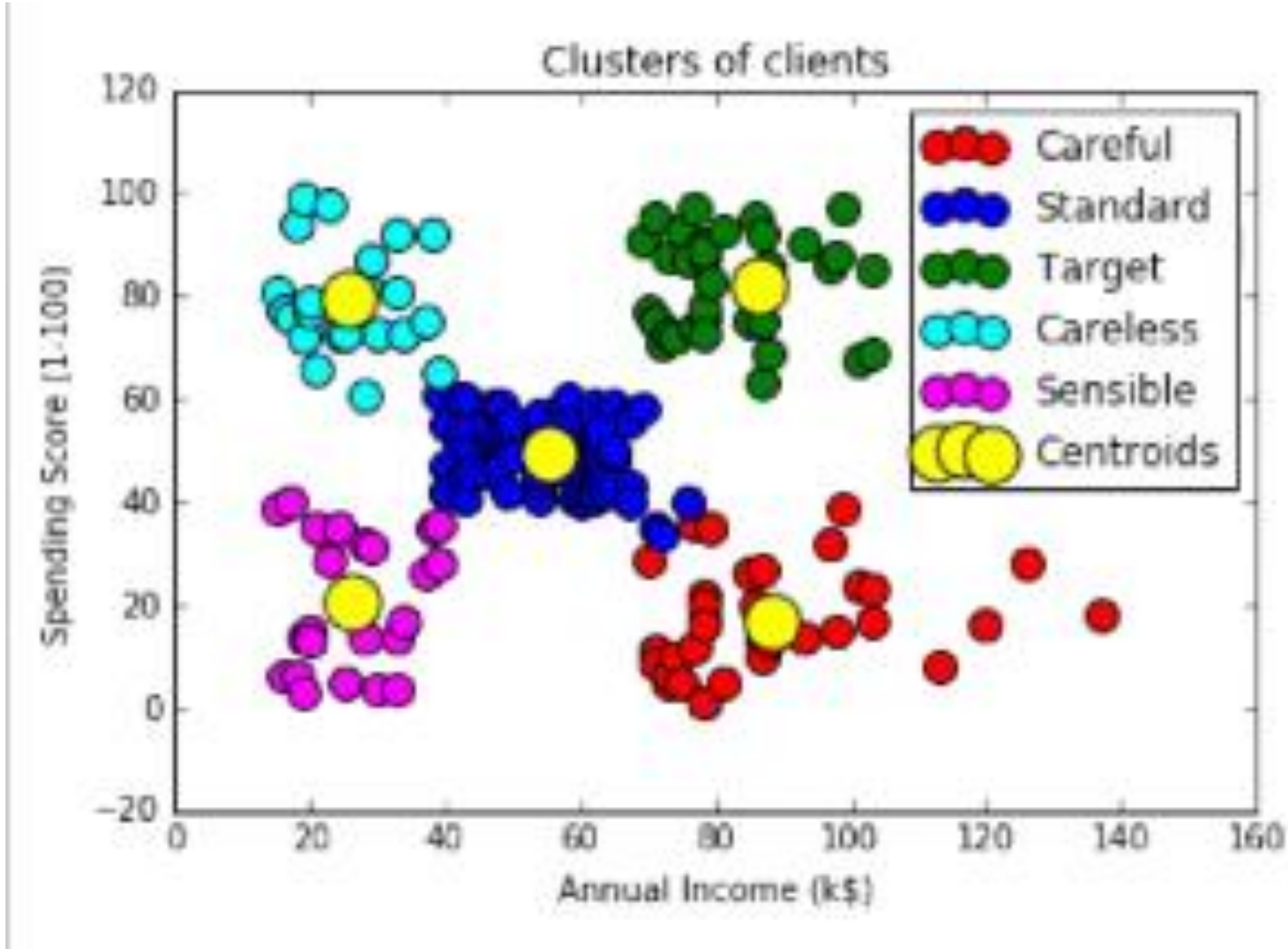


K-means clustering in python

```
# Apply K-Means to the mall dataset
kmeans = KMeans(n_cluster = 5, init ='k-means++', max_iter = 300,
n_init = 10, random_state = 0)
y_kmeans = kmeans.fit_predict(X)
```

K-means clustering in python

```
plt.scatter(X[y_kmeans==0, 0], X[y_kmeans ==0, 1], s=100, c='red', label = 'Careful')
plt.scatter(X[y_kmeans==1, 0], X[y_kmeans ==1, 1], s=100, c='blue', label = 'Standard')
plt.scatter(X[y_kmeans==2, 0], X[y_kmeans ==2, 1], s=100, c='green', label = 'Target')
plt.scatter(X[y_kmeans==3, 0], X[y_kmeans ==3, 1], s=100, c='cyan', label = 'Careless')
plt.scatter(X[y_kmeans==4, 0], X[y_kmeans ==4, 1], s=100, c='magenta', label = 'Sensible')
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s=300, c='yellow', label = 'Centroids')
plt.title('Cluster of Clients')
plt.xlabel('Annual Income K$')
plt.ylabel ('Spenting Score (1-100)')
plt.legend()
plt.show()
```


Clusters of clients
Careful
Standard
Target
Careless
Sensible
Centroids
Spending Score (1-100)
Annual Income (k$)
120
100
80
60
40
20
0
−20
0
20
40
60
80
100
120
140
160